ADDITIONAL DEVOPS INTERVIEW QUESTIONS – 4–5 YEARS

1️⃣ Linux / OS – VERY IMPORTANT

Q1: How do you check CPU, memory, and disk usage?

A:

"I use commands like top or htop for CPU and memory, free -m for memory, and df -h for disk usage. I also use du -sh to identify which directories are consuming disk space."

Example:

"For example, if a production server is running slowly, I check CPU and memory first using top and free -m. If disk usage is high, I use df -h and du -sh to find the directories consuming space."

Bash command:

```
top
free -m
df -h
du -sh /var/*
```

Q2: How do you troubleshoot a server with high CPU?

A:

"I first check CPU usage using top or htop, identify the process consuming CPU, check application logs, and verify whether the issue is traffic-related or application-related. Based on the cause, I may restart the affected process, scale the service, or involve the development team."

Example:

"For example, if an application process is consuming 95% CPU, I identify the process using top, check its logs, and determine whether high traffic or an application issue is causing it."

Bash command:

```
top
ps -eo pid,ppid,cmd,%mem,%cpu --sort=-%cpu | head
```

Q3: Disk is 100% full. What will you do?

A:

"I first confirm disk usage with df -h, then identify large files or directories using du. I check logs, temporary files, and old artifacts. I clean only unnecessary data, configure log rotation if required, and monitor the disk after cleanup."

Example:

"For example, if /var is consuming most of the disk, I check /var/log and old application logs, remove only unnecessary files, and configure log rotation to prevent the issue from happening again."

Bash command:

```
df -h
du -sh /var/*
du -sh /var/log/*
```

Q4: How do you check running processes?

A:

"I use commands such as ps -ef, top, or htop to identify running processes and their resource usage."

Example:

"For example, if I need to verify whether an application process is running, I use ps -ef and search for the process name."

Bash command:

```
ps -ef
ps -ef | grep java
```

Q5: How do you troubleshoot a service that is not running?

A:

"I check the service status using systemctl status, review service logs using journalctl, verify configuration and dependencies, fix the issue, restart the service, and confirm that it is healthy."

Example:

"For example, if Nginx is down, I check its status and logs, identify the configuration issue, fix it, restart Nginx, and verify that the service is running."

Bash command:

```
systemctl status nginx
journalctl -u nginx --since "30 minutes ago"
systemctl restart nginx
```

2️⃣ NETWORKING – VERY IMPORTANT

Q6: How do you troubleshoot connectivity between two servers?

A:

"I first check basic network connectivity using ping where applicable, then test the required port using telnet or nc. I verify DNS resolution, routing, firewall or security-group rules, and finally check application-level connectivity."

Example:

"For example, if an application server cannot connect to a database server on port 5432, I check connectivity, test port 5432, verify DNS, and then check firewall or security-group rules."

Bash command:

```
ping <server-ip>
nc -vz <server-ip> 5432
```

Q7: What happens when you type a URL in a browser?
A:
“First, DNS resolves the domain name to an IP address. Then the client establishes a network connection, usually TCP and TLS for HTTPS. The request reaches the load balancer or server, the application processes it, and the response is returned to the client.”
Example:
“For example, when a user accesses https://myapp.com, DNS resolves myapp.com, the HTTPS connection is established, the request reaches the load balancer, and the application returns the response.”
Bash command:
nslookup myapp.com
curl -v https://myapp.com

Q8: What is DNS?
A:
“DNS translates domain names into IP addresses so clients can locate services. For example, when a user accesses an application URL, DNS resolves that hostname to the required endpoint.”
Example:
“For example, myapp.com can resolve to an IP address such as 10.10.10.20, allowing the client to locate the application.”
Bash command:
nslookup myapp.com
dig myapp.com

Q9: What is the difference between public and private subnet?
A:
“A public subnet has a route to an Internet Gateway, while a private subnet does not directly accept inbound internet traffic. Application or database resources are commonly placed in private subnets for better security.”
Example:
“For example, I can place a load balancer in a public subnet and application/database servers in private subnets.”
Bash command:
ip route

Q10: What is the difference between Security Group and Network ACL?
A:
“A Security Group is a stateful firewall associated with resources such as EC2 instances. A Network ACL is a stateless firewall applied at the subnet level.”
Example:
“For example, I can allow application traffic to an EC2 instance using a Security Group, while Network ACL rules control traffic at the subnet level.”
Bash command:
aws ec2 describe-security-groups

3️ **TERRAFORM – MORE ADVANCED QUESTIONS**
Q11: What is Terraform state?
A:
“Terraform state keeps track of the infrastructure resources Terraform manages and their current known attributes. Terraform uses the state to determine what changes are required during the next plan or apply.”
Example:
“For example, if Terraform created an EC2 instance, its state records information about that resource so Terraform can determine whether it needs to create, update, or delete it later.”
Bash command:
terraform state list

Q12: Where do you store Terraform state?
A:
“For team environments, I prefer a remote backend such as an S3 backend, usually with appropriate state locking and access control, so multiple engineers can work safely.”
Example:
“For example, our team can store Terraform state in an S3 backend instead of keeping terraform.tfstate only on an engineer's local machine.”
Bash command:
terraform init
terraform state list

Q13: What is terraform plan?
A:
“terraform plan compares the desired configuration with the current state and infrastructure information and shows the changes Terraform intends to make without applying them.”
Example:
“For example, before adding a new EC2 instance, I run terraform plan to review what Terraform is going to create.”
Bash command:
terraform plan

Q14: What is terraform apply?
A:
“terraform apply executes the planned infrastructure changes and creates, updates, or deletes resources according to the Terraform configuration.”
Example:
“For example, after reviewing the plan and getting approval, I run terraform apply to create the required AWS resources.”
Bash command:
terraform apply

Q15: What happens if Terraform state is lost?
A:
“I would first check whether the state is stored in a remote backend and restore it from the available backup or versioning mechanism. I would avoid blindly recreating infrastructure and first verify the actual resources and state.”
Example:
“For example, if the local state file is deleted but the team uses a remote backend, I would verify the remote state and restore or recover it rather than recreating production resources.”
Bash command:
terraform state list
terraform refresh

Q16: How do you manage different environments in Terraform?
A:
“I use reusable modules and environment-specific variables or configurations. For example, development, staging, and production can use the same modules with different values while maintaining controlled state.”
Example:
“For example, the same EC2 module can be used for dev and production with different instance sizes, regions, or tags.”
Bash command:
terraform workspace list
terraform workspace select dev

Q17: What are Terraform modules?
A:
“Terraform modules are reusable groups of Terraform resources. They help standardize infrastructure and avoid duplicating the same configuration across environments.”
Example:
“For example, I can create an EC2 module once and reuse it for development, staging, and production.”
Bash command:
terraform init
terraform get

KUBERNETES – DEEP SCENARIO QUESTIONS
Q18: Pod is in Pending state. What do you check?
A:
“I check kubectl describe pod and review the events. I check whether there are sufficient CPU or memory resources, node availability, taints, tolerations, node selectors, PVC issues, or scheduling constraints.”
Example:
“For example, if a pod remains Pending because no node has enough CPU, I check node resources and either adjust resource requests or provide additional capacity.”
Bash command:
kubectl get pods
kubectl describe pod <pod-name>
kubectl get nodes

Q19: Pod is in CrashLoopBackOff. What do you do?
A:
“I check kubectl logs and kubectl describe pod. I look for application errors, missing environment variables, incorrect configuration, failed dependencies, or failed health checks. After fixing the root cause, I redeploy and verify the pod becomes healthy.”
Example:
“For example, if the application crashes because a database password environment variable is missing, I fix the configuration and redeploy the pod.”
Bash command:
kubectl logs <pod-name>
kubectl describe pod <pod-name>
kubectl get events --sort-by=.lastTimestamp

Q20: Pod is Running but application is not accessible. What do you check?
A:
“I verify the application process and container logs first. Then I check the readiness probe, Service selector, Service endpoints, port configuration, DNS, and Ingress or load-balancer configuration.”
Example:

“For example, the pod may be Running but not Ready because the application is listening on port 8080 while the Service is configured for the wrong port.”
Bash command:
kubectl get pods
kubectl logs <pod-name>
kubectl get svc
kubectl get endpoints

Q21: What is readiness probe vs liveness probe?
A:
“A readiness probe tells Kubernetes whether a container is ready to receive traffic. A liveness probe checks whether the container is still healthy and should be restarted if it becomes unhealthy.”
Example:
“For example, during application startup, readiness can prevent traffic from reaching the pod until the application is ready, while liveness can restart the container if it becomes unhealthy.”
Bash command:
kubectl describe pod <pod-name>

Q22: What happens when a Kubernetes node goes down?
A:
“Kubernetes detects the node failure and workloads managed by a Deployment or another controller can be rescheduled onto healthy nodes, provided sufficient capacity and scheduling conditions are available.”
Example:
“For example, if one worker node fails and there are replicas running on other nodes or enough healthy capacity, Kubernetes can reschedule affected workloads.”
Bash command:
kubectl get nodes
kubectl describe node <node-name>
kubectl get pods -o wide

Q23: How do you troubleshoot a Service that cannot reach a Pod?
A:
“I check the Service selector, endpoints, target pod labels, container port, readiness status, and network policies. I also test connectivity from another pod and check DNS resolution.”
Example:
“For example, if the Service selector is app=frontend but the pod label is app=front-end, the Service will not find the pod.”
Bash command:
kubectl get svc
kubectl describe svc <service-name>
kubectl get endpoints <service-name>
kubectl get pods --show-labels

Q24: What is a ConfigMap?
A:
“A ConfigMap stores non-sensitive configuration data separately from the container image, allowing configuration to be changed without rebuilding the application image.”
Example:
“For example, application environment values such as APP_ENV=dev can be stored in a ConfigMap and injected into the container.”
Bash command:
kubectl get configmap
kubectl describe configmap <configmap-name>

Q25: What is a Kubernetes Secret?
A:
“A Secret is used to store sensitive information such as passwords, tokens, or keys. In production, I prefer integrating Kubernetes with a dedicated secrets-management solution where appropriate.”
Example:
“For example, a database password can be stored as a Kubernetes Secret and injected into the application instead of hardcoding it in the Deployment YAML.”
Bash command:
kubectl get secrets
kubectl describe secret <secret-name>

---

5 DOCKER – DEEP QUESTIONS

Q26: What is the difference between Docker image and container?
A:
“A Docker image is an immutable package containing the application and its dependencies. A container is a running instance created from that image.”
Example:
“For example, myapp:1.0 is the Docker image, and when I run it with docker run, Docker creates a container from that image.”
Bash command:

docker images
docker run -d myapp:1.0
docker ps

Q27: How do you reduce Docker image size?
A:
“I use minimal base images, multi-stage builds, remove unnecessary packages and files, and use Docker layer caching effectively.”
Example:
“For example, instead of using a large Ubuntu image for the runtime, I can use a smaller runtime image and copy only the required application files.”
Bash command:
docker images
docker history myapp:1.0

Q28: What is a multi-stage Docker build?
A:
“It uses multiple stages in a Dockerfile. I can build the application in one stage and copy only the required output into a smaller runtime image, reducing the final image size.”
Example:
“For example, I can compile a Java application in a build stage and copy only the generated JAR into the final runtime image.”
Bash command:
docker build -t myapp:1.0 .
docker images myapp

Q29: Docker container is consuming high memory. What do you do?
A:
“I check container resource usage, application logs, and memory trends. I verify whether the application has a memory leak or incorrect limits. I may apply appropriate resource limits and involve the development team if the issue is application-related.”
Example:
“For example, if a container continuously increases memory usage, I check whether there is an application memory leak and temporarily apply an appropriate memory limit while investigating the root cause.”
Bash command:
docker stats
docker logs <container-id>

6️⃣ GIT – ADVANCED

Q30: Git merge vs rebase?
A:
“Merge combines branches and preserves their history with a merge commit when applicable. Rebase moves commits onto a new base and creates a cleaner linear history. I follow the team's branching policy and avoid rebasing shared branches unless it is explicitly agreed.”
Example:
“For example, I may rebase my local feature branch with the latest main changes before creating a Pull Request, if that is allowed by the team.”
Bash command:
git checkout feature
git fetch origin
git rebase origin/main

Q31: What is a Pull Request?
A:
“A Pull Request is a request to merge changes from one branch into another. It allows code review, automated testing, and approval before merging.”
Example:
“For example, after completing a feature, I push the feature branch and create a Pull Request from feature/login to main. Jenkins runs tests before approval.”
Bash command:
git push origin feature/login

Q32: How do you revert a bad production commit?
A:
“I identify the problematic commit, use git revert to create a new commit that reverses the change, test it, and merge it through the normal review and deployment process.”
Example:
“For example, if a production commit introduces an API issue, I use git revert, test the reverted version, and deploy it through the normal pipeline.”
Bash command:
git revert <commit-id>
git push origin main

7️⃣ JENKINS – ADVANCED

Q33: What is Jenkinsfile?
A:
“A Jenkinsfile defines the CI/CD pipeline as code. It contains stages such as checkout, build, test, security scan, image creation, deployment, and verification.”
Example:
“For example, our Jenkinsfile can contain stages for Git checkout, Maven build, unit tests, Docker image creation, image push, and Kubernetes deployment.”
Bash command:
cat Jenkinsfile

Q34: What are Jenkins pipeline stages?
A:
“Stages logically divide the pipeline into steps such as checkout, build, test, security scan, package, deploy, and post-deployment verification.”
Example:
“For example, a pipeline can have Checkout → Build → Test → Docker Build → Push → Deploy → Verify stages.”
Bash command:
mvn test
docker build -t myapp:1.0 .

Q35: How do you secure Jenkins credentials?
A:
“I store credentials in Jenkins Credentials or an integrated secrets manager instead of hardcoding them in the Jenkinsfile. Access is controlled based on roles and permissions.”
Example:
“For example, Docker registry credentials are stored in Jenkins Credentials and referenced by the pipeline instead of writing the username and password directly in the Jenkinsfile.”
Bash command:
docker login <registry>

Q36: Jenkins pipeline is stuck. What do you check?
A:
“I check the current stage, Jenkins agent availability, executor usage, external dependencies, credentials, network connectivity, and console logs to identify where the pipeline is waiting.”
Example:
“For example, if a pipeline is stuck waiting for an executor, I check Jenkins agents and executor availability before investigating the pipeline itself.”
Bash command:
ps -ef | grep jenkins

AWS – IMPORTANT 4-YEAR QUESTIONS
Q37: What AWS services have you worked with?
A:
“I have worked with services such as EC2, VPC, IAM, S3, CloudWatch, ECR, load balancers, and Kubernetes services such as EKS, depending on the project requirements.”
Example:
“For example, EC2 can host application workloads, S3 can store artifacts or backups, ECR can store Docker images, and EKS can run Kubernetes workloads.”
Bash command:
aws ec2 describe-instances
aws s3 ls
aws ecr describe-repositories

Q38: What is IAM?
A:
“IAM is AWS Identity and Access Management. It controls authentication and authorization through users, roles, policies, and permissions.”
Example:
“For example, an EC2 instance can use an IAM role to access an S3 bucket without storing AWS access keys on the server.”
Bash command:
aws iam list-roles
aws iam list-policies

Q39: Why use IAM roles instead of access keys?
A:
“I prefer IAM roles where possible because temporary credentials can be provided to AWS resources without storing long-lived access keys in code or configuration.”
Example:
“For example, an EC2 instance can assume an IAM role and access S3 without keeping permanent AWS access keys in a configuration file.”
Bash command:

aws sts get-caller-identity

Q40: EC2 server is unreachable. What do you check?
A:
"I check instance status, system and instance checks, security-group rules, subnet routing, network ACLs, public or private connectivity, load-balancer health if applicable, and operating-system level issues."
Example:
"For example, if an EC2 instance is unreachable over SSH, I check whether the instance is running, port 22 is allowed, the subnet route is correct, and the network ACL permits the traffic."
Bash command:
nc -vz <ec2-ip> 22
ssh -v ec2-user@<ec2-ip>

9️⃣ RELEASE / DEPLOYMENT QUESTIONS

Q41: What checks do you perform before production deployment?
A:
"I verify successful builds and tests, approved changes, configuration, image version, database migration requirements, deployment plan, rollback plan, and monitoring readiness."
Example:
"For example, before a production release, I verify the Docker image tag, Jenkins test results, Kubernetes manifest, database migration plan, rollback version, and monitoring dashboards."
Bash command:
kubectl apply --dry-run=client -f deployment.yaml
kubectl get deployment

Q42: How do you verify deployment after release?
A:
"I check deployment status, pod health, readiness, application logs, error rate, latency, and health-check endpoints. I also perform basic functional validation."
Example:
"For example, after deploying a new version, I check whether all pods are Ready, review application logs, and test the application health endpoint."
Bash command:
kubectl rollout status deployment/myapp
kubectl get pods
curl -f http://myapp/health

Q43: Deployment completed successfully but application is failing. What do you do?
"I compare the current version with the previous stable version, check logs and metrics, verify configuration and dependencies, and rollback if the impact is significant. After service restoration, I investigate the root cause."
Example:
"For example, Kubernetes shows the deployment as successful, but users receive HTTP 500 errors. I check application logs and recent configuration changes and rollback if required."
Bash command:
kubectl logs deployment/myapp
kubectl rollout undo deployment/myapp

Q44: How do you minimize deployment downtime?
"I use rolling deployments, readiness probes, health checks, load balancing, and controlled rollout strategies such as blue-green or canary deployments when appropriate."
Example:
"For example, during a rolling deployment, Kubernetes gradually replaces old pods with new pods while keeping healthy pods available to serve traffic."
Bash command:
kubectl rollout status deployment/myapp
kubectl get pods

🔟 DATABASE – DEVOPS LEVEL

Q45: How do you troubleshoot database connection issues?
"I check application logs, database availability, DNS, network connectivity, security-group rules, credentials, connection limits, and database health."
Example:
"For example, if an application cannot connect to PostgreSQL, I verify the database endpoint, DNS resolution, port 5432 connectivity, credentials, and security-group rules."
Bash command:
nc -vz <db-host> 5432
nslookup <db-host>

Q46: What happens if database connections are exhausted?
"I check active connections and application connection-pool settings. I look for connection leaks or sudden traffic increases and determine whether scaling or configuration changes are required."

Example:
“For example, if the application reports too many connections, I check active database connections and the application's connection pool to determine whether connections are not being released properly.”
Bash command:
psql -h <db-host> -U <user> -c "SELECT count(*) FROM pg_stat_activity;"

Q47: How do you handle database changes during deployment?
“I plan migrations carefully, test them in lower environments, take appropriate backups, and use backward-compatible migration strategies when possible so the application and database remain compatible during rollout.”
Example:
“For example, before adding a new database column required by a new application version, I test the migration in staging and make sure the old application version can continue working during the rollout.”
Bash command:
pg_dump -h <db-host> -U <user> -d <database> > backup.sql

1️⃣1️⃣ OBSERVABILITY – 4-YEAR LEVEL

Q48: What are the three pillars of observability?
“The three main pillars are metrics, logs, and traces. Metrics show system behavior numerically, logs provide detailed events, and traces help follow a request across distributed services.”
Example:
“For example, Prometheus can provide CPU and latency metrics, an ELK stack can provide application logs, and OpenTelemetry can provide distributed traces.”
Bash command:
curl http://localhost:9090/-/healthy
journalctl -u myapp

Q49: What is distributed tracing?
“Distributed tracing tracks a request as it moves across multiple services. It helps identify which service or dependency is causing latency or failure.”
Example:
“For example, a request may travel from API Gateway → authentication service → order service → payment service. Tracing helps identify which service caused the delay.”
Bash command:
curl -H "traceparent: <trace-id>" http://myapp/api/orders

Q50: How do you identify whether an issue is application or infrastructure related?
“I correlate application metrics, infrastructure metrics, logs, events, and recent changes. For example, if infrastructure metrics are normal but application errors increased after a deployment, I would investigate the application or configuration first.”
Example:
“For example, if CPU, memory, and node health are normal but HTTP 500 errors suddenly increase after a release, I investigate the application deployment and configuration.”
Bash command:
kubectl top pods
kubectl top nodes
kubectl logs deployment/myapp

1️⃣2️⃣ SRE / PRODUCTION – VERY IMPORTANT

Q51: What is SLI?
“SLI is a measurable indicator of service performance, such as availability, latency, or error rate.”
Example:
“For example, HTTP request success rate can be an SLI. If 99.8% of requests succeed, that value can be used to measure service reliability.”
Bash command:
curl -s -o /dev/null -w "%{http_code}\n" https://myapp.com/health

---

Q52: What is SLO?
“SLO is the target value for an SLI. For example, an availability SLO could be 99.9%.”
Example:
“For example, if our availability SLO is 99.9%, we expect the service to remain available for 99.9% of the defined measurement period.”
Bash command:
curl -f https://myapp.com/health

Q53: What is SLA?
“SLA is a formal agreement with a customer or business that defines expected service levels and usually includes consequences if those levels are not met.”
Example:
“For example, a customer contract may define a 99.9% monthly availability SLA for a production application.”
Bash command:
curl -s -o /dev/null -w "%{http_code}\n" https://myapp.com

Q54: What is error budget?
“An error budget represents the amount of unreliability allowed within an SLO. It helps teams balance reliability work with the speed of delivering new features.”
Example:
“For example, with a 99.9% availability SLO, the allowed downtime is approximately 0.1% of the measurement period. If the error budget is consumed quickly, the team may prioritize reliability work.”
Bash command:
uptime

1️⃣3️⃣ REAL-TIME SCENARIO QUESTIONS
Q55: Production is down and your manager asks for an update. What do you say?
“I would clearly communicate the current impact, what we have identified so far, what action we are taking, and when the next update will be provided. I would avoid guessing the root cause before confirming it.”
Example:
“For example: Production API is currently unavailable for some users. We identified failing pods and are checking the recent deployment. We are preparing a rollback and will provide the next update in 10 minutes.”
Bash command:
kubectl get pods
kubectl get events --sort-by=.lastTimestamp

Q56: Developer says “It works on my machine.” What do you do?
“I compare the environments, including application version, dependencies, configuration, environment variables, and runtime versions. I use logs and reproducible builds to identify the difference.”
Example:
“For example, if the application works locally but fails in Kubernetes, I compare the Docker image, environment variables, Java/Python version, configuration, and external dependencies.”
Bash command:
docker image inspect myapp:1.0
env

Q57: Deployment works in Dev but fails in Production. Why?
“I check environment-specific configuration, secrets, permissions, networking, dependencies, resource limits, and deployment manifests. I compare the environments systematically rather than assuming the application code is the issue.”
Example:
“For example, the same image may work in Dev but fail in Production because a production secret is missing or the production database is not reachable.”
Bash command:
kubectl get configmap
kubectl get secrets
kubectl describe pod <pod-name>

Q58: Your deployment caused errors. What is your first priority?
“My first priority is to minimize user and business impact. If rollback is the safest option, I roll back to the last known stable version, verify recovery, and then investigate the root cause.”
Example:
“For example, if HTTP 500 errors increase immediately after a deployment, I rollback to the previous stable version and verify that error rates return to normal.”
Bash command:
kubectl rollout undo deployment/myapp
kubectl rollout status deployment/myapp

Q59: You made a production mistake. What do you do?
“I immediately communicate the impact, take the safest action to restore service, document what happened, and perform RCA. I also add preventive controls so the same mistake is less likely to happen again.”
Example:
“For example, if I accidentally deploy an incorrect configuration, I immediately inform the team, rollback the change, verify service recovery, and later add validation checks to prevent the same mistake.”
Bash command:
kubectl rollout undo deployment/myapp
kubectl rollout status deployment/myapp

Q60: You are asked to perform a task you have never done before. What do you do?
“I first understand the requirement, check the official documentation and existing team procedures, test the approach in a lower environment, and then implement it safely. If there is risk, I involve a senior engineer before making production changes.”
Example:
“For example, if I am asked to configure a new AWS service that I have not worked with before, I first understand the requirement, test it in Dev, validate the configuration, and only then plan the Production implementation.”
Bash command:
aws sts get-caller-identity
terraform plan